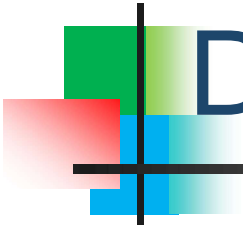


Dynamic memory allocation

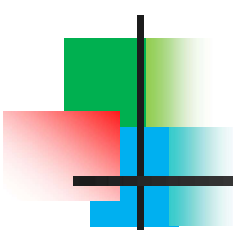
Using malloc()



Dynamic Memory Allocation

- So far, memory allocation was handled automatically at compile time.
- Certain cases you don't know how much memory to set aside
 - Dynamically allocate memory to variables at run-time.
 - Example is an unsized array.



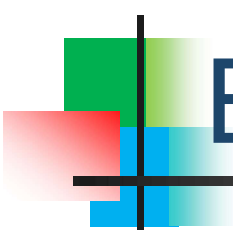


malloc()

- It is used to allocate memory at run time
- Syntax: **void *malloc(size_t size)**
 - Accepts a single argument, the number of bytes you want to allocate dynamically
 - If successful, returns a void pointer to the first allocated byte of memory
 - Return a NULL pointer if unsuccessful
- The function prototype is given in `stdlib.h`
- Should use casting to convert the void * to the specific type

*If you want space for 10 integers, and an integer (int) is 4 bytes, you need to request $10 * 4 = 40$ bytes. The best practice is to use the `sizeof()` operator : `malloc(10 * sizeof(int));`*



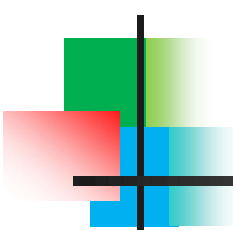


Ex: malloc()

```
// 1. ALWAYS check if malloc() failed  
if (ptr == NULL) {  
    printf("Memory allocation failed!\n");  
    return 1; // Exit the program  
}
```

```
// 2. IMPORTANT: When you are done, you MUST free the memory  
free(ptr);
```

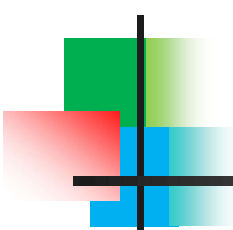
- Create an integer pointer and allocate 10 integer memory locations
 - Read the elements of the array using the pointer
 - Print the elements using the pointer



Ex: Dynamic 2D arrays

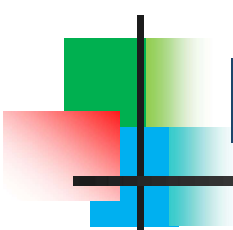
- Write a C program to create and print a 2D floating point array of size *row* x *col*, where *row* and *col* are user input.
 - Input rows and cols
 - Allocate the row pointers
 - For each row pointer, allocate the column
 - Input the array elements
 - Print the array elements





Ex: Using free()

- Write a function that read n sequence of characters and prints them in reverse order. The value of n is a parameter of the function.

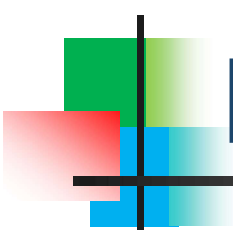


Importance of free()

- Memory leakage: space allocated within function is lost when function call ends without freeing up the memory
 - No way to figure out where the memory is

```
void fun() {  
    float *nums;  
    int N = 5;  
  
    nums = (float *) malloc(N * sizeof(float));  
} /* function ends without call to free() */
```





Different type of pointers

- pointer to a specific data type
- wild pointer
- null pointer
- void pointer
- **dangling pointer**



A dangling pointer is a pointer that holds the address of a memory location that is no longer valid. This usually happens because the memory it used to point to has been freed or deleted.

Dangling pointer

- A pointer pointing to a memory location that has been deleted (or freed)

- `int *ptr=(int *) malloc(sizeof(int));`

malloc(sizeof(int)): This function allocates (reserves) a small block of memory, just big enough to hold one integer.

- `free(ptr); // ptr becomes a dangling pointer`

Pointer 3

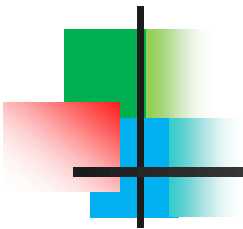
free(ptr): This function deallocates the memory that ptr is pointing to. It tells the operating system, "I am finished with this block of memory; you can use it for something else now."

Deleted Object

After the free(ptr); line, the variable ptr still holds the same address as before. However, that address is no longer valid. The memory it points to has been "deleted," as the diagram shows.

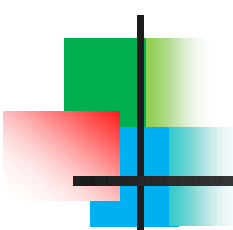
A common way to prevent this specific problem is to set the pointer to NULL immediately after freeing it:

```
free(ptr);  
ptr = NULL;
```



Dynamic memory allocation

Using calloc()

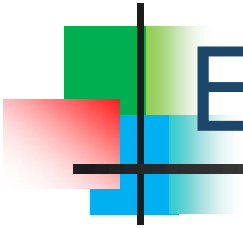


calloc()

*Using calloc(): `int *ptr = (int *)calloc(10, sizeof(int));`
This reserves 40 bytes.
Crucially, all 10 integers in this block are automatically set to 0.*

- Another function to dynamically allocate memory
- Syntax: **`void *calloc(size_t n, size_t size)`**
 - Requires two arguments,
 - First argument is number of elements
 - Second argument is size of element
 - If successful, returns a void pointer to the first allocated byte of memory
 - Returns a NULL pointer if unsuccessful

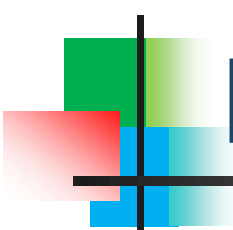




Ex: calloc()

- Create an integer pointer and allocate 10 integer memory locations
 - `int *p;`
`p = (int *) calloc(10 , sizeof(int));`
- Check if the memory allocation failed
 - `if (p==NULL)`
`{`
`printf("Memory allocation failed\n");`
`exit(1);`
`}`





Ex:

- Write a program that accepts n floating point numbers from the user and prints the largest element.
[use `calloc()` for dynamic memory allocation]

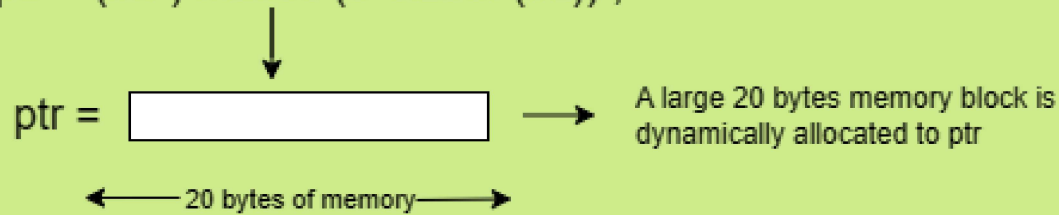


malloc

Malloc ()

```
int* ptr = (int*) malloc (5* sizeof (int)) ;
```

4 bytes

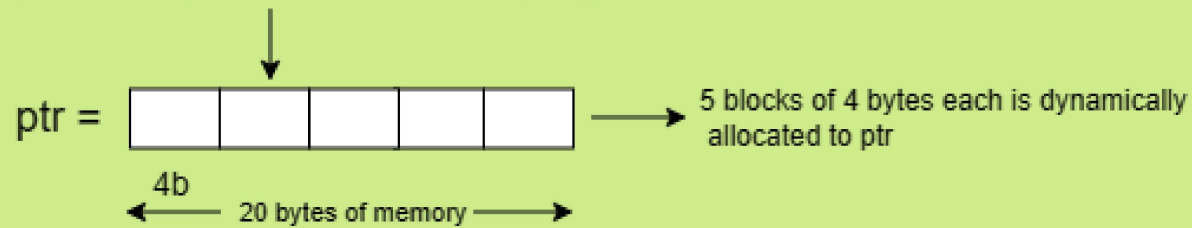


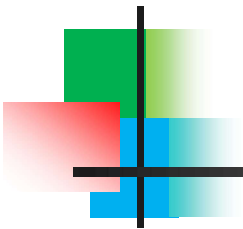
calloc

Calloc ()

```
int* ptr = (int*) calloc (5, sizeof (int)) ;
```

4 bytes



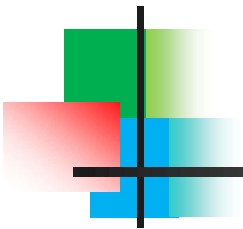


malloc vs. calloc

- Allocates memory in bytes
- Allocated space is not initialized

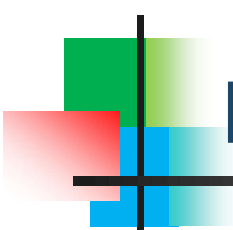
- Allocates as a sequence of bytes
- Allocated space initialized to zero.





Dynamic memory allocation

Using `realloc()`



realloc()

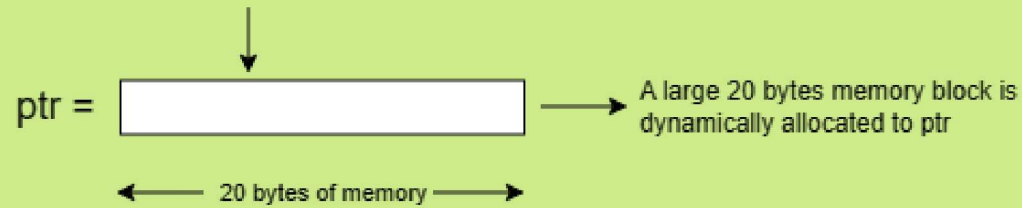
- Allows to allocate more/less memory space without losing data.
- Syntax: **void *realloc(void *ptr, size_t newsize)**
 - Accepts two arguments
 - First is a pointer to the first byte of memory that was previously allocated
 - Second is the new size of the block which may be smaller or larger than original size



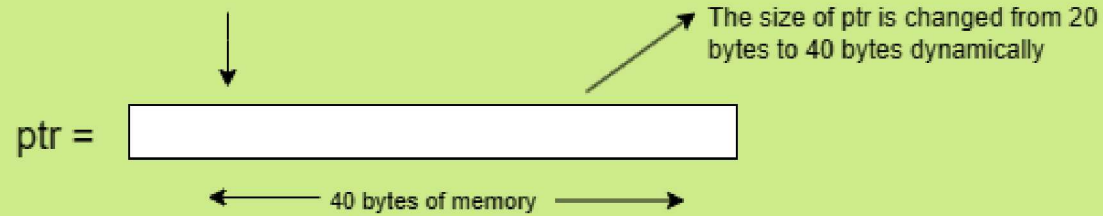
Realloc()

```
int* ptr = (int*) malloc (5* sizeof (int));
```

4 bytes



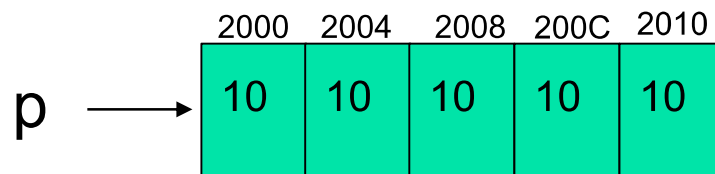
```
ptr = realloc (ptr, 10* sizeof (int));
```



Ex: realloc()

- Create an integer pointer and allocate 5 integer memory locations

- `int *p; p = (int *) malloc(5*sizeof(int));`



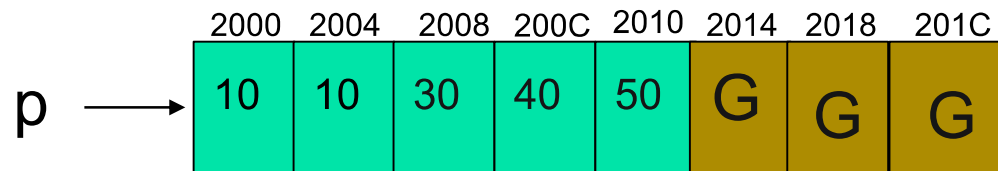
- Increase the size of allocated memory to 3 more integers

- `p = (int *)realloc(p, 8 * sizeof(int));`

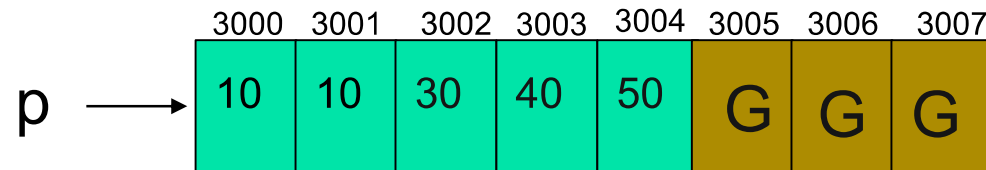
Outcome of realloc()

Three cases

- Case I: If sufficient memory available after block

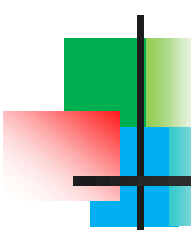


- Case II: Else, allocate in some other location



- Case III: No memory available, return NULL





Ex: Expanding array

- Write a program that accepts an initial number of floating point numbers from the user and later allows the user to expand the array to add more elements.
[use `realloc()` for dynamic memory allocation]

